Reasoning Commitment Points in Transformer Chain-of-Thought

A Research Development Report: From Hypothesis to Experiment

Aditya Singh · IIT Kharagpur (23ME3EP03)

Supervised by Prof. D. Pham, University of Birmingham

Period: May–June 2026 · **Status:** Work in Progress

> *This document is a complete record of the intellectual journey behind the CoT Commitment Points research project. It covers the theoretical genesis, every design decision, every failure, every bug, and every result — in the order they happened. It is written for two audiences: Prof. D. Pham, who needs the full scientific record, and Adi, who needs to understand what he built, what it cost, and what it is actually worth.*

Part I: The Question That Started Everything

The Theoretical Gap Nobody Had Filled

The project did not begin with an experiment. It began with an observation about what a theorem does and does not say.

Merrill and Sabharwal (2024) proved that constant-depth transformers without chain-of-thought are bounded by TC^0 — a circuit complexity class shallow enough that it cannot solve graph connectivity problems, cannot count arbitrarily, cannot simulate sequential automata. A single forward pass, no matter how many parameters, is computationally bounded. This is a clean and striking result: the architecture that powers every large language model in production is formally incapable of a wide class of problems if asked to answer immediately.

But chain-of-thought changes the picture completely. With O(n) intermediate generation steps, the model escapes TC^0 and reaches context-sensitive expressivity. With polynomial steps, it

reaches exactly P — all polynomial-time-solvable problems. The escape mechanism is the generation of intermediate tokens; each token is a new forward pass, and the chain of forward passes composes into computation that no single pass could perform.

This is the aggregate result. The total computation across a CoT trace is sufficient for polynomial-time reasoning.

Here is what the theorem does not say: it is completely silent on **how that computation is distributed across the trace**. It says the total is enough. It says nothing about whether the critical computation happens in the first step or the last, whether it is front-loaded or evenly distributed, whether an incorrect early step propagates irrecoverably or remains correctable.

This gap is not a minor omission. Consider what happens empirically when a large language model starts on a wrong reasoning path. It almost never self-corrects. It compounds the initial error across subsequent steps, arriving at a wrong answer with apparent confidence. The theory says the total computational budget is sufficient — but the model is spending it on compounding an error rather than recovering from one. At what specific point in the trace does the wrong answer become locked in? Is there a stage at which correct computation can still be injected to override the wrong path, and a stage after which it is too late?

This is a causal question, not an observational one. And no existing paper had answered it.

The Third RASP Failure Mode

The theoretical framework used in the project's presentation material separated two failure modes using the RASP diagnostic framework. The RASP language maps directly onto transformer operations. If no RASP program exists for a task, the model faces an architectural failure — the computation is beyond what the architecture can express. If a RASP program exists but the model fails, it faces a training failure — the capability is there but wasn't learned.

The project's research question implicitly introduces a third failure mode that neither of these captures: **localized computational failure**. The model has the RASP-equivalent computation available. It attempts to execute it during the Setup and Computation stages. But it executes it incorrectly — makes an arithmetic error, misunderstands a constraint, takes a wrong approach. By the time it reaches the Transition stage, the wrong result has propagated through the residual stream. The final answer is already committed internally, even though the surface text has not yet announced it.

This is not architectural failure. This is not training failure. It is a precise and localized failure in a specific phase of the trace, after which the wrong answer is structurally unrecoverable even under direct causal intervention on the model's internal state.

This third mode is a genuine conceptual contribution. It tells you something specific about where to intervene: before the computation phase is complete, not after. It tells you something specific about what process reward models and steering tools are fighting against when they try to correct wrong reasoning.

The experiment was designed to locate this third failure mode empirically.

Part II: The Original Design and Its Systematic Collapse

What Was Planned

The original proposal was coherent and well-scoped. Every design decision had a clear rationale:

- **Model:** Qwen2.5-1.5B-Instruct. Small enough to fit in memory without quantization, large enough to produce multi-step reasoning chains, fast enough to generate 160 traces in a reasonable time.
- **Library:** TransformerLens. The standard tool for mechanistic interpretability, providing clean access to residual stream activations at every layer via the `cache["resid_post", layer_i]` interface.
- **Dataset:** GSM8K hard tail (problems 900–1319). 419 problems, the portion of the GSM8K test set where the model fails most frequently under greedy decoding. Standard benchmark, verified ground truth, structured reasoning chains.
- **Pairs:** 160 total — 80 correct (greedy decode, T=0) + 80 incorrect (temperature-sampled, T=1.0). Within-problem pairing: the same question, the same model, different paths.
- **Measurement:** Teacher-forced logit inspection. Run the full wrong trace through the model, inject correct activations at stage-labeled token positions, read the logit at the answer-token position, check if it shifted toward the correct answer.
- **Stages:** Four independent semantic stages: Setup (problem restatement), Computation (arithmetic operations), Transition (logical connectors like "therefore"), Conclusion (final answer and boxed result).
- **Timeline:** Four weeks. One Kaggle session per week.

Every single component of this design needed to change. Not one survived contact with reality.

Death One: TransformerLens Cannot Handle NF4

The first thing that died was the library choice. TransformerLens works by wrapping PyTorch's `nn.Linear` modules into custom `WrappedLinear` objects at model load time. This wrapping is what enables the hook registration system — when you call `model.run_with_cache()`, it captures residual stream activations at every layer through these hooks.

BitsAndBytes NF4 quantization also wraps `nn.Linear` — into `Linear4bit` objects. These two wrappers conflict at the module level. TransformerLens cannot reliably register hooks on BnB-quantized layers. The result is not a clean error message. It is silent hook failures — the code runs without crashing, but the hooks do not fire, and the activation cache is empty or corrupted. You would run an entire experiment, get numbers that look plausible, and have no idea you were measuring nothing.

The diagnosis was clear once the module hierarchy was inspected: `model.model.layers[0].self_attn.q_proj` was a `Linear4bit` object, not a `WrappedLinear` object. TransformerLens had not successfully wrapped it.

The fix was to abandon TransformerLens entirely and use raw PyTorch forward hooks:

```
model.model.layers[i].register_forward_hook(hook_fn)
# output[0] is always the residual stream: (batch, seq_len, d_model)
```

This is approximately 30 lines of code. It requires no additional library. It works regardless of quantization method because it hooks the module's output, which is always computed in float16 regardless of weight precision. It gives semantically identical access to the residual stream as TransformerLens's cache.

The loss is the clean API. The gain is a system that actually works.

Death Two: Device Splits Break Patching

With quantization came a memory problem. The first attempt loaded Qwen2.5-7B in float16 (15.2GB) with `device_map="auto"`. PyTorch saw 32GB of total VRAM across two T4s and split the model: layers 0–13 on cuda:0, layers 14–27 on cuda:1.

For activation patching, this is fatal. When you store activations from the correct trace at layer 10, they live on cuda:0. When you try to inject them at layer 10 during the wrong-trace forward pass, the hook fires correctly — but when you try to inject stored activations from the wrong layer (say layer 20, which lives on cuda:1), the tensors are on different devices. Every cross-device injection either crashes with a device mismatch error or requires an explicit `.to(device)` call that adds overhead and complexity to every single hook.

The fix was NF4 quantization pinned to a single GPU. At 4-bit precision, a 14B-parameter model uses approximately 7–10GB depending on double quantization settings. The 14B model

loaded at 9.98GB on cuda:0, leaving 6GB free for activation storage and KV cache during generation. cuda:1 stayed at 0GB.

This also meant upgrading from 1.5B to 14B, which turned out to be necessary anyway for an unrelated reason.

Death Three: GSM8K Is Too Easy for 14B

The most damaging failure took the longest to diagnose. The original plan assumed a model with ~70–75% accuracy on GSM8K — a reasonable assumption for 1.5B. But the upgrade to 14B changed everything. Qwen2.5-14B achieves approximately 90–95% accuracy on GSM8K, including on the "hard tail" (problems 900–1319). The model solves these problems so reliably that finding organic wrong traces via temperature sampling became essentially impossible.

The initial attempt used T=0.8. The 10-sample diagnostic showed 10/10 correct predictions. The temperature was raised to T=1.2. The diagnostic showed 1/25 wrong traces. At T=1.4, slightly better but still less than 10% wrong traces.

It was not a matter of temperature. The model simply does not make meaningful reasoning errors on GSM8K at this scale. What few "wrong" predictions appear are almost always stochastic final-token errors: the model reasons correctly through the entire trace and then emits a wrong number at the `#### [answer]` position, seemingly by chance. The reasoning chain is intact and correct; only the generated answer token is wrong.

This distinction is critical. A wrong trace built on stochastic final-token errors is useless for studying commitment. The model's internal state — the residual stream at the answer position — reflects the correct reasoning upstream. In a teacher-forced forward pass over this "wrong" trace, the model's logit at the answer position already strongly predicts the correct answer, even before any patching. There is nothing to recover because the model was never committed to being wrong internally.

This is the **already-flipped problem**: the wrong trace is wrong in text but correct in internal state.

Death Four: Teacher-Forced Logit Inspection Has a Fundamental Artifact

Even if a dataset with genuine wrong traces were found, the measurement method was wrong.

The plan was: run the full wrong trace through the model, inject correct activations at stage-labeled positions, check the logit at the answer-token position. The problem is teacher-forcing. During this forward pass, the model sees every token in the wrong trace,

including all future tokens after the injection point — including the wrong answer token itself. The logit at the answer position reflects the full sequence context. The model can "attend" to its own wrong answer token downstream of the injection point, effectively reading the answer from the context rather than computing it from the patched activations.

This is not a subtle effect. It means that a stage that produces a logit shift might be doing so because the model is cheating off future wrong tokens, not because the patched activations are genuinely redirecting the computation. Conversely, a stage that produces no shift might produce no shift because future context overwhelms the patch, not because the stage carries no causal information.

The teacher-forcing artifact makes every teacher-forced logit measurement ambiguous. A non-zero shift is uninterpretable because it conflates "patched activations did useful work" with "model attended to wrong future context and found patched activations insufficient to override it."

This is the artifact that motivated the switch to Truncate and Generate.

Part III: The Collection Odyssey

Six strategies. Five failures. This section is the heart of the engineering story.

Strategy One: Temperature Sampling at T=0.8

Simple. Expected to work. Did not work. Zero wrong traces in ten samples across five problems. The diagnostic output: all 10 predictions correct. Moved on.

Strategy Two: Hard Tail at T=1.2–1.4

Identified problems 900–1319 as the "hard tail" — the most difficult GSM8K problems. Ran the diagnostic again with T=1.2–1.4. Result: 1/25 wrong traces. Still not viable for collection at 10%+ error rate needed. The underlying model-capability problem was now clear: temperature was not the lever.

Strategy Three: Greedy-Failure Collection (Inverted Condition)

Strategically sound. If temperature sampling cannot reliably induce errors, collect problems where the model's MAP prediction (greedy decode) itself fails. These are problems where the

model is genuinely wrong, not just occasionally wrong. Use the greedy-wrong output as the wrong trace, then sample at low temperature to get correct traces.

The code:

```
def collect_pairs_v3(ds_hard, target_pairs, greedy_budget):
    candidates = []
    for item in ds_hard:
        greedy_out  = gen(make_prompt(q), temperature=0.0)
        greedy_pred = extract_gt(greedy_out)

        # BUG: this condition is INVERTED
        if greedy_pred is None or not answers_match(greedy_pred, gt):
            continue  # SKIPS failures, KEEPS successes

        candidates.append({"wrong": greedy_out, ...})
```

The intended logic: skip problems where greedy is CORRECT (useless, model already got it right), keep problems where greedy FAILED (these are the interesting wrong traces). The actual logic: skip problems where greedy FAILED, keep problems where greedy SUCCEEDED. The `not` inverted the condition.

The output confirmed the error but was easy to misread:

```
Found: GT=15, greedy=15 ✓
Found: GT=1,  greedy=1  ✓
Found: GT=70000, greedy=70000 ✓
```

These ✓ symbols indicated greedy success. They were being interpreted as "found a valid pair" when they were in fact showing that the code had incorrectly flagged correct greedy predictions as "failures."

Three full sessions ran on this strategy before the bug was caught. Each session collected pairs from easily-solved problems, Phase 2 attempted temperature sampling on those problems, found no wrong traces (because the model is nearly certain on these problems), and the overall wrong collection produced zero valid pairs.

The fix is two characters:

```
# Before (broken):
if greedy_pred is None or not answers_match(greedy_pred, gt):
    continue  # wrong: skips failures

# After (correct):
if greedy_pred is None or answers_match(greedy_pred, gt):
    continue  # correct: skips successes, keeps failures
```

Strategy Four: Injection-Based Wrong Traces

With the collection strategy stuck, the injection approach was introduced as a fallback. Injection: take a correct trace, replace only the final answer token (change `#### 18` to `#### 23`), leave all reasoning intact. Guaranteed wrong answer, instantly generated, no sampling needed.

This solved the collection problem and introduced a contamination problem.

When the experiment ran on injection pairs, the already-flipped pre-check caught 7/8 pairs. Here is why: an injection pair has correct reasoning throughout and only the final answer token modified. In a teacher-forced forward pass, the model attends to all the correct upstream computation. The logit at the answer position reflects that correct computation and strongly predicts the correct answer, regardless of what is physically in the sequence at that token. The pair is "already flipped" in internal state, even before patching.

The one pair that escaped the pre-check and produced results was revealing: the patching was trivially easy. The correct-trace activations and the wrong-trace activations at every stage are identical (because the reasoning is the same text), and patching them into each other produces high flip rates everywhere — not because the stages carry different causal loads, but because you are patching the correct reasoning into itself and it trivially produces the correct answer.

Injection was abandoned. Every injection pair in the experiment produced meaningless measurements.

Strategy Five: Greedy-Failure Collection (Fixed)

With the inverted condition fixed, genuine greedy failures were found. But a new problem emerged: the already-flipped rate was 7/8.

The error magnitudes explained why:

```
GT=64800, greedy=60    (1080× error)
GT=250,   greedy=5     (50× error)
GT=300,   greedy=150   (2× error)
```

These are not reasoning errors. A model that reasons correctly about a word problem and arrives at a 50× wrong answer is not making a reasoning mistake — it is outputting a random number at the answer token against the grain of its own computation. In teacher-forced inference, the correct reasoning upstream overrides the wrong final token. Already flipped.

The one pair that survived — GT=1, greedy=2 — showed a small off-by-one error that was likely a genuine reasoning failure. This pair produced the cleanest commitment curve seen in the project: setup +45.8 Δld, computation +12.4 Δld, transition +1.2 Δld, conclusion ~0. The

step function in one data point.

But a dataset built entirely on near-identity errors is too narrow and too fragile. The underlying problem remained: GSM8K hard tail problems still do not produce structural reasoning failures in 14B.

Strategy Six: MATH Level 4-5

The dataset switch that finally worked. MATH Level 4-5 (DigitalLearningGmbH/MATH-lighteval) contains competition mathematics problems requiring multi-step symbolic reasoning: algebra, number theory, combinatorics, geometry. Qwen2.5-14B achieves approximately 40% accuracy on Level 4-5 — meaning 60% of problems produce genuine greedy failures, and within those, temperature sampling at T=1.2–1.4 produces organically wrong traces at a viable rate.

The pair orientation switched:

- **Correct trace:** greedy decode (T=0). On problems the model solves correctly, greedy output is deterministic and noise-free.
- **Wrong trace:** temperature sampling at T=1.2–1.4 until an incorrect answer appears. On MATH Level 4-5, the model follows a genuinely wrong reasoning path from early in the trace — takes a wrong approach, misapplies a theorem, makes a structural algebraic error — and the wrong answer is the conclusion of internally consistent but wrong computation.

The critical difference from stochastic-token failures: the wrong trace on a MATH Level 4-5 problem has wrong intermediate computations, wrong intermediate results, and wrong final answer. The residual stream accumulates wrong information throughout the trace. In a teacher-forced forward pass, the model's logit at the answer position reflects all that wrong upstream computation and correctly predicts the wrong answer.

The already-flipped rate dropped from approximately 80% (GSM8K) to approximately 21% (MATH Level 4-5). Collection became viable.

Part IV: The Infrastructure Rebuilt From Scratch

From TransformerLens to Raw Hooks

The final hook infrastructure uses Python's native `torch.nn.Module.register_forward_hook`. The design:

```
_acts = {}  # layer_idx → (seq_len, d_model) tensor, stored on CPU

def mk_store_hook(i):
    def hook(module, input, output):
        hidden = output[0]  # always the residual stream
        _acts[i] = hidden.detach().squeeze(0).cpu()
    return hook

def mk_inject_hook(i, patch_positions, source_acts):
    def hook(module, input, output):
        if i not in source_acts: return output
        h   = output[0]
        src = source_acts[i].to(h.device)
        h2  = h.clone()
        for pos in patch_positions:
            if pos < h2.shape[1] and pos < src.shape[0]:
                h2[0, pos] = src[pos]
        return (h2,) + output[1:]
    return hook
```

`output[0]` is the residual stream for every Qwen2.5 transformer layer. This was verified empirically — the Test 1 diagnostic in the first successful session confirmed: shape=(1, 17, 5120), dtype=torch.float16, device=cuda:0, consistent across all 48 layers.

The Test 2 and Test 3 sanity checks were run before any experiment: zeroing layer 14's residual stream produces a logit delta of 23.77 (hooks are causal), and the full store→inject round-trip is deterministic (reproducibility delta = 0.000000). These are the foundational validations. Without them, every subsequent result is suspect.

Truncate and Generate: The Core Methodological Contribution

The teacher-forcing artifact required a fundamental redesign. The solution: instead of running the full wrong trace and inspecting logits, **cut the wrong trace at the stage boundary and actually generate the continuation from that truncation point**.

The design:

1. Tokenize the full wrong trace: `[prompt + wrong_reasoning + wrong_answer]`

2. Identify the truncation point: the last token index belonging to the target stage

3. Run a baseline generation from `[prompt + wrong_reasoning[:truncation_point]]` — no patching. The model generates the continuation. If this baseline already produces the correct answer, the pair is not committed at this stage and is excluded.

4. Store correct-trace activations via hooks during a forward pass over `[prompt + correct_reasoning]`

5. Run the patched generation: inject correct-trace activations at the stage-labeled token positions during the prefix forward pass. Generate the continuation.

6. Record whether the generated continuation contains the correct answer.

The critical property: there are no future wrong tokens in the prefix during generation. The model generates autoregressively from the patched prefix. It must compute the answer from scratch, using whatever information the patched activations have provided. This eliminates the teacher-forcing artifact entirely.

The already-flipped pre-check falls out naturally: if the baseline generation (step 3) already produces the correct answer, the pair is excluded. No ambiguity, no logit threshold.

This design is methodologically closer to Bogdan et al.'s Thought Anchors (which replace sentences and resample continuations) than to ROME-style logit patching — except instead of replacing surface text, it replaces residual stream activations. The causal chain is clean: if patching computation-stage activations changes the generated answer from wrong to correct, those activations are causally necessary for the correct trajectory at this truncation point.

The Cumulative Stage Architecture

The T&G; design forced a change in how stages are defined. Independent stage patching — patch ONLY computation tokens, leaving everything else intact — cannot be achieved with truncation. When you truncate at the last computation token, everything before that point (including all setup tokens) is in the prefix. The prefix is fixed. You are not choosing which tokens to "include" — you are choosing where to cut.

This created the **cumulative** stage design:

- `pre_setup`: truncate after the first 10% of trace tokens
- `setup`: truncate after the last setup-labeled token
- `computation_only`: truncate after the last computation-labeled token, patch only computation tokens
- `+computation`: truncate at max(setup ∪ computation) tokens, patch both setup and computation
- `+transition`: truncate at max(setup ∪ computation ∪ transition) tokens, patch all three

The `computation_only` condition is the cleanest: it truncates immediately after the last computation token and patches only those tokens, giving genuine stage isolation.

The `+computation` condition has a structural flaw. If any setup-labeled tokens appear after computation tokens (common in MATH solutions, where a phrase like "Let's verify" appears after the main algebraic computation), the truncation point extends to encompass those late

setup tokens — potentially including the wrong `\boxed{answer}` in the prefix if `\boxed{}` is misclassified as setup. The model then generates from a prefix containing the literal wrong answer. This explains the non-monotonicity: `computation_only` outperforms `+computation` not because setup tokens are harmful, but because the truncation artifact puts the wrong answer in the context.

Decision reached after the last run: drop everything except `computation_only`. All other conditions are either degenerate (already-flipped rate >70%), confounded (+computation with `\boxed{}`), or statistically identical to `+computation` (+transition adds zero marginal computation-labeled tokens in MATH solutions).

Part V: The Bug Register

Every one of these bugs would have produced a paper with false results if undetected. In the order they were found and fixed.

Bug 1: max_new_tokens=150 — Silent Data Killing

What it was: In both `run_experiment` and `run_per_layer`, the generation calls were hardcoded to `max_new_tokens=150`.

What it did: After truncating at the setup-stage boundary, the model needs to generate 500–1500 tokens to reach the answer on MATH Level 4-5 problems (full computation + conclusion + `\boxed{answer}` or `#### answer`). Capping at 150 means the generation ends mid-reasoning. `extract_gt` returns None. The pair is dropped, logged as "already_flipped" or "skip." Effective: the setup and pre_setup conditions lost most of their data silently.

Impact: Every setup=0% result in all pre-fix runs is suspect. The null result that the original paper was building its narrative around may be entirely a generation-cutoff artifact, not a genuine finding about setup-stage causal neutrality.

Found by: AI-assisted code audit (Gemini IDE, June 2026).

Fix: Replace with `max_new_tokens=MAX_NEW_TOKENS` (set to 2048 in config).

Bug 2: Inverted Control Metric

What it was: `run_experiment` used `answers_match(patched_pred, gt)` to compute "flip" for BOTH the main experiment AND the control group (patching wrong activations into correct

traces).

What it did: For the main experiment (correct acts → wrong trace), matching GT = successful flip = correct. For the control (wrong acts → correct trace), matching GT means the model RETAINED the correct answer despite wrong-act injection = NOT a flip. The control axis was inverted: a perfectly robust model would show 100% "flips," a completely corrupted model 0%.

Impact: The correct-group control in every pre-fix run measured the inverse of its intended interpretation. The "100% retention" that was celebrated as methodology validation was in fact measuring... 100% retention, but through a coincidentally-inverted sign. The number happened to be correct (100% retention = 100% "flips" with the wrong metric, but 0% "flips" with the right metric and 100% retention is verified by 0% failures). This particular bug happened to produce a number that was interpretable — but only by accident.

Found by: AI-assisted code audit (Gemini IDE, June 2026).

Fix: Target-aware metric: `is_flip = not answers_match(patched_pred, gt)` for control experiments.

Bug 3: Inverted Phase 1 Collection Condition

What it was: `if greedy_pred is None or not answers_match(greedy_pred, gt): continue`

What it did: Skipped greedy FAILURES (the interesting cases), kept greedy SUCCESSES (useless for wrong-trace collection).

Impact: Three full sessions ran on this strategy. All collected pairs came from problems the model solves reliably, making organic wrong-trace sampling impossible. The session time was wasted on a collection loop that would never succeed.

Found by: Manual inspection after noticing output showed `greedy=GT` ✓ for all "found" pairs.

Fix: Remove `not`: `if greedy_pred is None or answers_match(greedy_pred, gt): continue`

Bug 4: extract_gt First Match Instead of Last

What it was: `re.search(r'####\s*...', text)` finds the FIRST occurrence of the `####` pattern.

What it did: MATH traces contain intermediate computation steps like `16 - 3 = 13` that can match the answer extraction pattern before the final `#### 42` at the end. Pair 0's wrong trace extracted GT=13 when the actual wrong answer was `\boxed{something}` appearing later.

Impact: Wrong pairs were collected with incorrect `wrong_val` fields. The already-flipped check compared against the wrong value, letting contaminated pairs through or incorrectly dropping valid ones.

Fix: `re.findall(...)[-1]` to get the last `####` occurrence. Also added `\boxed{}` extraction using `text.rfind('\\boxed{')` for the LAST occurrence.

Bug 5: \boxed{} Classified as Setup Instead of Conclusion

What it was: PATTERNS["conclusion"] did not include `r'\\boxed\{'`. MATH solutions end with `\boxed{42}`. Without this pattern, lines containing `\boxed{}` fell through to the residual category (setup) or matched computation patterns (number + operator patterns inside the braces).

What it did: The wrong answer's `\boxed{}` token was classified as setup. In the `+computation` cumulative condition, the truncation point is `max(setup_tokens ∪ computation_tokens)`. If a setup-labeled `\boxed{wrong_answer}` token appears after all computation tokens, the truncation point extends past it. The prefix fed to the generator contains `\boxed{wrong_answer}` explicitly. The model is being asked to generate a continuation after reading its own wrong answer. Recovery is nearly impossible.

Impact: Explains the `computation_only > +computation` non-monotonicity. What looked like a mechanistic finding (setup activations actively interfere with computation patching) was a truncation artifact. The `\boxed{}` bug was likely the largest source of confounding in the experimental results.

Found by: Mathematical analysis of the truncation design — specifically, examining what token the `+computation` truncation point was landing on for sample pairs.

Fix: Add `r'\\boxed\{'` to PATTERNS["conclusion"]. Critically: two backslashes in the raw string (`r'\\boxed\{'`), which regex reduces to one literal backslash. The four-backslash version (`r'\\\\boxed\{'`) compiles without error but matches the string `\\boxed{` (double backslash before `boxed`), which never appears in tokenizer.decode() output. This is a bug that silently does nothing.

Bug 6: Duplicate Pairs in Top-Up Collection

What it was: `collect_pairs_v3` always starts from the beginning of `ds_hard`. When loading 19 pre-collected pairs from file and running top-up to reach N=40, the top-up re-scans from problem index 0 and can re-collect questions already in the loaded pairs.

Impact: Effective N is reduced by the number of duplicates. In the worst case, the experiment runs on duplicate problem pairs where the "different" traces for the same problem may have

correlated errors, violating the independence assumption.

Fix: `existing_questions = {p["question"] for p in pairs}` and `extra = [p for p in extra if p["question"] not in existing_questions]` after top-up collection.

Bug 7: ds_hard Undefined When Top-Up Triggers

What it was: If `pairs.json` already exists, the notebook loads it instantly and skips the data-loading cell that defines `ds_hard`. The top-up block (`if len(pairs) < N_PAIRS`) then calls `collect_pairs_v3(ds_hard, ...)` and hits `NameError: name 'ds_hard' is not defined`.

Impact: The top-up crashes immediately, no pairs are collected beyond the loaded set. In normal top-to-bottom execution this does not manifest (Cell 13 defines ds_hard before Cell 17), but any out-of-order execution triggers it.

Fix: Guard: `if 'ds_hard' not in dir() or ds_hard is None: [reload ds_hard]`

Bug 8: greedy_budget=600 for Top-Up

What it was: The collection function was called with `greedy_budget=600` for the top-up operation.

What it did: Each MATH problem generation takes ~50s. 600 problems × 50s = 500 minutes = 8+ hours just for Phase 1 scanning. This would kill a Kaggle session before reaching any patching experiments.

Fix: `greedy_budget=300`. At ~25% overall collection efficiency (60% greedy correct × ~40% yield on organic wrong traces), 300 problems yields ~75 candidate pairs — more than sufficient for the 21 additional pairs needed.

Bug 9: Double-Backslash Regex Convention Ambiguity

What it was: Uncertainty about whether `r'\\boxed\{'` (two backslashes) or `r'\\\\boxed\{'` (four backslashes) correctly matches `\boxed{` in tokenizer.decode() output.

The math: In Python raw strings, each `\\` is two literal characters. Regex then reduces each `\\` to one literal backslash. So `r'\\boxed\{'` = regex that matches `\boxed{` (one backslash). `r'\\\\boxed\{'` = regex that matches `\\boxed{` (two backslashes). `tokenizer.decode()` produces text with single backslashes. Therefore `r'\\boxed\{'` is correct, `r'\\\\boxed\{'` is wrong.

Why it's in the bug register: The four-backslash version compiles without error, runs without error, and silently matches nothing. If applied incorrectly, the "fix" looks applied but has no effect. All experimental runs would continue with the confound active, undetected.

Fix: Mandatory pre-session gate check:

```
sample = next((p["wrong"] for p in pairs if "boxed" in p["wrong"]), None)
m2 = re.search(r'\\boxed\{', sample)
m4 = re.search(r'\\\\boxed\{', sample)
assert m2 is not None  # two-backslash must match
assert m4 is None      # four-backslash must NOT match
```

Part VI: The Control Architecture Built From Nothing

The original proposal had zero controls. The final design has four. This is the biggest methodological difference between what was proposed and what was built.

Control One: Correct-Group (Methodology Validation)

Patches correct-trace activations into correct-trace forward passes. The correct trace is already correct. The patched activations are from a correct trace. Expected result: 100% retention at every stage — the model produces the correct answer with or without patching because everything is already correct.

If this control fails — if retention drops below 100% — the hooks are corrupting the forward pass. Nothing in the main experiment is interpretable until this is fixed.

Result: 100% retention across all stages in every run after the control metric bug was fixed. This is the foundational validation. Every subsequent result rests on it.

Control Two: Cross-Problem Null (Specificity Validation)

Patches a DIFFERENT problem's correct-trace activations into the wrong trace. The source activations are from a correct solution to a completely unrelated problem. The question: does problem-specificity matter? If the 50% flip rate in `computation_only` is measuring "this problem's correct computation activations override wrong computation," cross-problem patching should give near-zero — because a different problem's computation activations don't contain the relevant information.

If cross-problem gives similar results to within-problem, the measurement is detecting something generic: residual stream disruption of sufficient magnitude causes flips regardless of what's being injected. That would make the experiment scientifically uninteresting.

Result: 0% at +computation (0/17), 21.4% at computation_only (3/14).

The gap at +computation is total — perfect specificity. The gap at computation_only is partial but large: 50% within-problem vs 21.4% cross-problem, a 28.6 percentage point difference at N=12 with overlapping confidence intervals. The cross-problem 21.4% is based on 2–3 events at n=14 and may be noise. The core claim — within-problem activations are substantially more effective than cross-problem activations — is directionally confirmed. N=40 will determine whether this survives with non-overlapping intervals.

Control Three: Shuffled-Positions (Position Specificity)

Injects same-problem correct activations at randomly shuffled token positions instead of computation-labeled positions. Same problem, same activations, random placement. Expected: if computation-stage positions are specifically important, structured patching should outperform random.

Result: shuffled sometimes outperformed structured at +computation (47% vs 35%). This is the unsettling finding. It means computation-stage-labeled positions are not uniquely optimal. However, the cross-problem control (which was not run for shuffled positions) would almost certainly show near-zero, consistent with the within-problem activations being problem-specifically informative even when placed randomly. The story: problem identity matters strongly; precise position identity within the problem matters partially but not exclusively. Computation-labeled positions are a sufficient set of positions, not the uniquely optimal set.

Control Four: Block D (Deleted)

Proposed to test NF4 depth-accumulation artifacts: if early-layer patches accumulate quantization noise through subsequent NF4 layers while late-layer patches don't, the per-layer curve's gradient might be a quantization artifact rather than genuine layer-level commitment signal.

Ran with N=5. Produced results ranging from 0% to 60% per layer. Standard error of 20% per data point. Completely uninformative. The control was valid in concept and invalid in execution at this sample size. Deleted from the final session plan.

Part VII: The Results That Emerged

N=2 Under the Old Methodology (Invalid)

The first numbers anyone saw: setup +14.6 Δld, computation +13.9 Δld, transition +0.3 Δld, conclusion ~0. These were from the original teacher-forced logit inspection methodology on N=2 injection pairs.

These numbers have since been superseded by everything that followed. The methodology was wrong (teacher-forcing artifact), the pairs were contaminated (injection), and N=2 supports no statistical claim. They are included in the technical report's Section 5 only with explicit disclaimer.

N=19 Under T&G; on MATH Level 4-5 (The Actual Pilot)

After the full methodology overhaul — T&G;, MATH Level 4-5, organic pairs, correct controls — the cleaned results at N=19:

Condition	Flip Rate	95% CI	n
pre_setup	0.0%	[0%, 0%]	5
setup	0.0%	[0%, 0%]	4
computation_only	57.1%	[29%, 79%]	14
+computation	35.3%	[18%, 59%]	17
+transition	35.3%	[18%, 59%]	17

Controls at computation_only:

- Cross-problem: 21.4% (3/14)
- Correct-group: 100% (9/9)
- Shuffled: 28.6% (4/14)

These numbers require careful interpretation.

`computation_only = 57.1%` vs `cross-problem = 21.4%`: the core result. Computation-stage activations from the paired correct trace are substantially more effective at redirecting wrong trajectories than activations from an unrelated correct trace. The gap is 35.7 percentage points. At N=14 the intervals are wide but the direction is clear.

`setup = 0.0%` and `pre_setup = 0.0%`: these must be treated with caution. The max_new_tokens=150 bug may have caused most setup-stage pairs to be dropped via the

already-flipped filter (generation cut off before `####`, extract_gt returned None, pair excluded). The 0% result is not yet confirmed as a genuine null. It requires the bug fix and re-running.

`+computation = +transition = 35.3%`: these are empirically identical. The same 6 pairs flip in both conditions. This is not a finding about transition tokens carrying no additional information — it is a segmentation artifact. MATH solutions' final answer section is classified as conclusion, not transition, meaning adding transition to the patch set adds no tokens and the truncation point does not move.

The Per-Layer Curve (N=12, Directional)

Layer	Flip Rate
0	33.3%
4	33.3%
8	25.0%
12	41.7%
16	33.3%
20	33.3%
24	41.7%
28	25.0%
32	25.0%
36	8.3%
40	8.3%
44	0.0%
47	0.0%

The curve is not what was predicted. The hypothesis, informed by Dutta et al.'s functional rift, was that a specific layer band (approximately layers 12–28) would carry 70%+ of the signal, with negligible contribution from early and late layers. A clean, localizable commitment circuit.

What emerged instead: a roughly flat plateau from layers 0–32 at 25–42%, dropping sharply at layers 36–40, and falling to zero at layers 44–47. The signal is not localized. It is broadly and apparently redundantly distributed across the first 32 layers, then abruptly absent in the final 16 layers.

This is consistent with the residual stream functioning as a communication bus, where information is written and read by multiple layers, accumulating gradually rather than residing in

a specific circuit. Any single layer in 0–32 can partially redirect the trajectory — because all of them have access to the computation-stage information through the shared residual stream. But after layer 32, the information has been consolidated into a form that single-layer intervention cannot override.

The drop at layers 36–47 is the actual commitment signature in the per-layer analysis. It is saying: by the time computation reaches the deep processing layers, it has been converted into a representation that individual patching cannot reverse.

At N=12, this is directional. The flat plateau could be noise. The sharp drop at 36 could be a statistical artifact of 1–2 fewer events. N=30 (the target for the final session) will either confirm or kill this interpretation.

Part VIII: What It All Means

The Step Function, Not a Curve

The original hypothesis expected a smooth S-curve: low flip rate at setup, rising through computation and transition, perhaps plateauing at conclusion. The commitment point would be the inflection — where the curve bends sharply from recoverable to unrecoverable.

What emerged is not a curve. It is a step function. Zero at setup/pre_setup. High (57%) at computation_only. Unchanged at transition. Then zero again in the late layers of the per-layer analysis.

This is a sharper and more specific claim than the original hypothesis. It says: **there is a discrete phase transition between computation and what follows it**. The computation phase produces and encodes the committed answer. Everything after it is announcement, not decision.

The Thought Anchors Tension and Its Resolution

Bogdan, Macar, Nanda and Conmy (2025) found that planning and uncertainty-management sentences have disproportionate causal impact on final answers, while arithmetic sentences — the computation stage — often do NOT. This appears to directly contradict `computation_only = 57%`.

The resolution is methodological, not empirical.

Bogdan measures importance by replacing visible text sentences and resampling continuations. If replacing the arithmetic sentence "20 − 5 = 10" with a different sentence does not change the final answer, they conclude that sentence is not causally important.

This project measures importance by replacing residual stream activations at computation-labeled token positions. If injecting correct-trace activations at those positions changes the generated answer, those activations are causally important.

These measure different things. A model can write the arithmetic sentence "20 − 5 = 10" as surface text while the actual computation has already been committed in the residual stream before those tokens are written. The surface text is causally inert (Bogdan) because it is a post-hoc transcription of computation already done. The residual states at those tokens are causally important (this project) because they carry the committed computation, accessible for intervention.

Turpin et al. (2023) and Lanham et al. (2023) establish that CoT explanations can be post-hoc announcements of decisions already encoded in activations — that the visible text and the internal state can diverge. The Thought Anchors finding and this project's finding are both consistent with this: "therefore" text is causally important at the surface (it signals reasoning structure to the model's own continuation), while computation text is causally important at the activation level (it carries the committed answer in the residual stream).

These are not contradictions. They are two measurements of the same phenomenon from different methodological angles.

Implications for Steering and Monitoring

If computation-stage commitment is confirmed at N=40:

Process reward models. Lightman et al. (2024) train step-level verifiers that score individual CoT steps and provide RL signals at the step level. If computation-stage commitment is real and complete before transition tokens are generated, a process reward model operating at transition or conclusion tokens is providing feedback on an already-committed answer. The feedback signal arrives too late to redirect the reasoning. PRMs need to operate during or immediately after the computation phase — not after the model starts writing "therefore."

Activation steering. Any steering intervention designed to redirect wrong reasoning must target early-to-mid generation positions, specifically during the computation phase. Steering at transition or conclusion tokens is post-commitment and cannot work.

Real-time monitoring. A practical system that wants to flag wrong reasoning in progress must evaluate the quality of the setup and computation — the approach being taken and the arithmetic being executed — rather than watching for internal consistency in the conclusion. By

the time the model writes a confident wrong conclusion, the answer has been committed for potentially hundreds of tokens.

Part IX: The Meta-Lesson in Execution

This section is uncomfortable. It belongs in the document.

The Numbers

The project ran for approximately 6 weeks. N=12 usable data points for the main finding at the close of the last analyzed session. The ratio of analysis-to-execution in this project was approximately 20:1 — twenty diagnostic documents, analysis sessions, and review rounds for every experimental run that produced new data.

Every analysis was correct. Every diagnosis was accurate. Every pivot was justified. None of that is the issue.

The issue is the gap between "this is what needs to happen" and "this happened." Specific instances:

- The per-layer analysis was identified as necessary in roughly session 6 of the project. It ran for the first time in session 22. That is 16 sessions of correctly identifying the gap without closing it.
- The T&G; pivot was theoretically motivated before the already-flipped problem had fully manifested. It was implemented approximately 8 sessions later.
- The `max_new_tokens` bug was introduced in session ~4. It was found in session ~18. Every experiment in between produced setup=0% results that may have been entirely artifactual.

The Pattern

Each session produced a correct diagnosis of what was wrong. The correct response to that diagnosis was to make a one-line fix and run the notebook. The actual response was to produce another analysis document about the diagnosis, review the diagnosis, discuss the diagnosis, audit the diagnosis, and then (sometimes) make the fix and queue the run for the next session.

The documentation in this project is exceptional. The understanding of what was going wrong, and why, and what would fix it, was consistently accurate. The translation of that understanding

into running code happened at a fraction of the pace it should have.

The Lesson

In experimental ML research, **the unit of progress is a running experiment**. A diagnosis that does not produce a running experiment within 24 hours is overhead. A correct analysis that is followed by another analysis is debt. The correct ratio of analysis to execution is heavily execution-weighted — approximately one analysis session for every three execution sessions.

This project ran the opposite ratio. The lesson for every subsequent project: when you know what to fix, fix it and run it. The notebook will tell you what the next problem is. No analysis document has ever been as informative as the output of an actual run.

Part X: Honest Contribution and Where This Lands

What This Project Contributes

The project is, at its core, a causal experiment. It does not observe correlations between stage labels and answer correctness. It intervenes: it forces a change in the model's internal state and measures whether the final answer changes. This causal framing distinguishes it from probing studies (Afzal et al.), faithfulness studies (Lanham et al., Turpin et al.), and text-manipulation studies (Bogdan et al.).

Specifically, it contributes:

The first wrong→correct CoT paired patching experiment. Mehrafarin et al. patch CoT→direct-answer. This patches wrong-CoT→correct-CoT. Different direction, different question, complementary findings.

Stage-level granularity. Prior token-level patching (Mehrafarin) asks "which token positions carry signal?" This asks "which semantic stage contains the committed computation?" The semantic stage framing is coarser but more interpretable and directly connects to the TC^0 theoretical framework.

Cross-problem null control validating specificity. Many activation patching studies do not include a cross-problem control. Without it, a positive result could reflect generic residual stream disruption. The 0% cross-problem result at +computation and the 21% vs 50% gap at computation_only confirm that what is being measured is problem-specifically committed computation, not generic signal.

The per-layer curve. At N=12 directional: broadly distributed signal in layers 0–32, absent in layers 36–47. This is a partial mechanistic characterization that no prior stage-level study provides.

What It Cannot Claim

At current N: **none of the specific numbers can be quoted**. The 57.1% at computation_only has a 95% CI of [29%, 79%]. A 50-percentage-point interval is not a result — it is a direction. The cross-problem gap of 28.6pp has overlapping intervals. The per-layer curve at N=12 has ±20pp standard error per layer.

The setup=0% result: **unverified**. The max_new_tokens bug may have artifactually produced this result. Until re-run with the fix, setup=0% is not a finding.

The "transition is too late" claim: **cannot be made**. +transition = +computation because the segmentation produces the same truncation point. No information about transition-stage causal neutrality can be extracted from these results.

Where It Lands

For MATS/MAIA/ARENA fellowship applications: Strong. The project demonstrates the ability to identify a real research question, connect it to formal theory, build an experiment from scratch, catch methodological failures before they produce wrong papers, and maintain intellectual honesty about what has and has not been established. Fellowship committees care about reasoning quality and research trajectory. This project shows both, even without definitive results.

For arXiv: After N=40 with non-overlapping cross-problem CIs. Not before.

For a workshop paper: After N=40, per-layer at N=30, and the setup null verified with the max_new_tokens fix. The `computation_only` vs `cross-problem` gap, if confirmed with non-overlapping CIs at N=40, is a publishable point result for an MI workshop.

For a main-track conference: This project needs multi-model replication (at minimum one other model family), a verified null result at setup (pending the max_new_tokens fix), and a cleaner per-layer story. That is 4–6 more weeks of experimentation. The current project is the foundation for that, not the submission itself.

Appendix A: The Bug Register — Summary Table

ID	Bug	Would Have Produced	How Found	Fix
1	max_new_tokens=150	setup=0% as false null finding	AI audit	MAX_NEW_TOKENS=2048 everywhere
2	Inverted control metric	Incorrect 100% retention interpretation	AI audit	Target-aware metric logic
3	Inverted Phase 1 condition	3 sessions of zero-yield collection	Output inspection	Remove `not`
4	extract_gt first match	Wrong wrong_val, invalid already-flipped check	Pair 0 inspection	re.findall(...)[-1]
5	\boxed{} not in conclusion patterns	Truncation artifact, false non-monotonicity	Stage analysis	r'\\boxed\{' in PATTERNS["conclusion"]
6	Duplicate pairs in top-up	Correlated pairs, inflated effective N	Code review	Deduplication by question string
7	ds_hard undefined on top-up	NameError crash, collection failure	Code review	Existence guard
8	Four-backslash regex	Silent no-op fix, unfixed confound	Mathematical analysis	Two-backslash + gate check
9	greedy_budget=600	8-hour collection session, session killed	Timing calculation	greedy_budget=300

Appendix B: The Complete Design Evolution

Component	Original	Final	Why Changed
Model	Qwen2.5-1.5B	Qwen2.5-14B-Instruct NF4	TL incompatibility, then GSM8K accuracy
Library	TransformerLens	Raw PyTorch hooks	NF4 wrapper conflict
Dataset	GSM8K hard tail	MATH Level 4-5	14B too accurate on GSM8K
Pair type	Greedy + temperature-sampled	Greedy-correct + temperature-sampled-wrong	Already-flipped problem

Measurement	Teacher-forced logit	Truncate and Generate	Teacher-forcing artifact
Stages	4 independent	computation_only	Cumulative design forced by T&G; others degenerate or confounded
Controls	None	Correct-group + cross-problem + shuffled	Reviewability, specificity
Metric	∆ld + flip rate	Flip rate (primary)	T&G; generates text, not logits
N	160 (target)	19 (achieved)	Collection difficulty, bug overhead

---

Appendix C: The Reading List

See the companion Reading List document for annotated references covering the theoretical foundations (TC^0/P theory), mechanistic interpretability methodology, mechanistic CoT studies, faithfulness literature, and positioning relative to prior work.

Document last updated: June 2026. Contact: Aditya Singh, IIT Kharagpur (aditya26189@kgpian.iitkgp.ac.in). GitHub: github.com/Aditya26189. Supervised by Prof. D. Pham, University of Birmingham.

This is a work-in-progress research document. Full results and statistical analysis pending the N=40 experimental session.